



Chapitre 4:

Les sous-programmes

I. Introduction

En langage C, **les sous-programmes** (appelés **fonctions**) permettent de **diviser un programme complexe en plusieurs parties plus petites, réutilisables et faciles à comprendre.**

Cette approche s'appelle la **modularité** et elle facilite la conception, le débogage, la maintenance et la réutilisation du code.

Les fonctions sont des parties de code source qui permettent de réaliser le même type de traitement plusieurs fois et/ou sur des variables différentes.

I. Introduction

En C, les sous-programmes s'appellent des fonctions ou des procédures.

Une fonction a toujours un type de retour, qui correspond au type du résultat qu'elle peut renvoyer et qui peut être n'importe lequel des types que nous avons précédemment étudié.

Le type de retour peut être **void** si on souhaite que la fonction (procédure) ne renvoie rien.

Une fonction a aussi un nom et une liste de zéro, un, ou plusieurs, paramètres.

II. Déclaration

Syntaxe :

```
TypeDeRetour  nom_fonction (Type1 param1, Type2 param2)
{
    /* variables locales */

    /* instructions de la fonction */

    return ValeurDeRetour; /* optionnelle si la
                           fonction ne retourne rien (de type void) */
}
```

III. Les prototypes

Le prototype d'une fonction est une description d'une fonction qui est définie plus loin dans le programme. On place donc le prototype au début du programme (avant la fonction principale `main()`).

Le prototype d'une fonction reprend **exactement** l'en-tête (header) de la fonction, mais pas son corps, qui est remplacé par un point-virgule.

Cette description permet au compilateur de « vérifier » la validité de la fonction à chaque fois qu'il la rencontre dans le programme.

III. Les prototypes

Syntaxe :

```
TypeDeRetour  nom_fonction (Type1 param1, Type2 param2);  
Ou TypeDeRetour  nom_fonction (Type1, Type2);
```

Exemples :

```
# include <stdio.h>  
int somme (int a, int b); /* déclaration du prototype  
                           de la fonction somme */  
  
void main () {  
    int x, y, z;  
    //...  
    z = somme (x, y);  
    //...  
}  
/* implémentation du corps de la fonction somme */  
int somme (int a, int b) {  
    // ...  
}
```

IV. Passage des paramètres

Tout paramètre **non fonctionnel** d'une fonction peut être de type ordinaire (int, float,...) ou de type pointeur.

Dans le cas d'un paramètre ordinaire : le passage de ce dernier est appelé **passage par valeur**. Dans ce cas la fonction fait une copie de la valeur du paramètre. Elle n'utilise que la copie, qui peut être modifiée. Mais à la fin du traitement de la fonction, la valeur de la variable passée en paramètre n'a pas été modifiée, puisque c'est seulement la copie qui a été modifiée.

Exemple :

```
void add (int a, int b, int c) {  
    c = a + b;  
}
```

IV. Passage des paramètres

```
#include <stdio.h>
```

```
void add (int a, int b, int c) {  
    c = a + b;  
}
```

```
void main() {  
    int x = 5, y = 7, z = 3;  
    printf("Avant l'appel de 'add':\n  
        \tx = %d, y = %d, z = %d\n", x, y, z);  
    add(x, y, z);  
    printf("Après l'appel de 'add':\n  
        \tx = %d, y = %d, z = %d\n", x, y, z);  
}
```

IV. Passage des paramètres

Dans le cas d'un paramètre de type pointeur : le passage de ce dernier est appelé **passage par adresse**. Dans ce cas la fonction ne fait pas de copie de la valeur du paramètre, comme c'est fait dans le cas du passage par valeur. Par conséquent, si le paramètre change de valeur, alors à la sortie de la fonction, la variable passée en paramètre contient la dernière modification, donc la dernière valeur.

Exemple :

```
void add (int a, int b, int *c) {  
    *c = a + b;  
}
```

V. Appels des fonctions

Une fonction est appelée comme étant une instruction du programme si elle ne retourne rien, sinon elle est appelée dans une expression d'une affectation, comparaison ou autre.

Exemple:

```
#include <stdio.h>
float carre (float x) {
    x = x * x;
    return x;
}
void main() {
    float a = 2, b;
    b = carre(a);
    /* la valeur de a ne change pas après
    l'appel de la fonction carre */
    printf("Le carre de %f est = %f", a, b);
}
```

V. Appels des fonctions

Exemple:

```
#include <stdio.h>
void permuter (float *x, float *y) {
    float aux;
    aux = *x;
    *x = *y;
    *y = aux;
}
void main() {
    float a = 2.3, b = 3.2;
    permuter(&a, &b);
    /* la valeur de a sera 3.2 et la valeur de
       b sera 2.3 */
    printf("a = %f et b = %f", a, b);
}
```

VI. Les fonctions d'ordre supérieur

Les fonctions d'ordre supérieur représentent un ensemble de concepts avancés qui facilitent la manipulation des fonctions en tant que données. Cette capacité offre deux aspects fondamentaux :

1. Paramètre Fonctionnel : Ces fonctions peuvent accepter d'autres fonctions en tant que paramètres, offrant ainsi une approche puissante pour créer des fonctions **génériques** capables de travailler avec diverses logiques métier.
2. Type de Retour Fonctionnel (**n'est pas supporté par le langage C**) : Ces fonctions ont la capacité de retourner une fonction comme résultat, permettant ainsi de créer des mécanismes flexibles et dynamiques au sein du programme.

VI. Les fonctions d'ordre supérieur

Les fonctions d'ordre supérieur apportent une puissante abstraction au paradigme de programmation fonctionnelle. Leur utilité peut être comprise à travers plusieurs aspects :

1. Abstraction et Modularité :

Les fonctions d'ordre supérieur permettent de créer des abstractions plus élevées en encapsulant des fonctionnalités spécifiques. Cela améliore la modularité du code en isolant les détails d'implémentation.

2. Réutilisabilité du Code :

En passant des fonctions en tant que paramètres, on peut écrire des fonctions génériques qui acceptent diverses opérations. Cela favorise la réutilisabilité du code, car une même fonction peut être utilisée avec différentes logiques.

VI. Les fonctions d'ordre supérieur

Exemple 1:

```
#include <stdio.h>
```

```
void appliquer(int a, int b, int (*operation)(int, int))
{
    printf("Résultat : %d\n", operation(a, b));
}
```

```
int somme(int x, int y) { return x + y; }
int produit(int x, int y) { return x * y; }
```

```
void main() {
    appliquer(3, 4, somme);    // somme est passée comme
    paramètre fonctionnel
    appliquer(3, 4, produit); // produit aussi
}
```

VI. Les fonctions d'ordre supérieur

Exemple 2:

```
/* Définition d'un type de fonction */
typedef int (*operation) (int, int);

int addition (int a, int b) { return a + b; }
int multiplication (int a, int b) { return a * b; }

void appliquer (operation oper , int a, int b) {
    printf("Résultat = %d\n ", oper(a, b));
}

void main() {
    appliquer (addition, 3, 4);          /* Affiche 7 */
    appliquer (multiplication, 3, 4);    /* Affiche 12 */
}
```